The Ultimate Data Structures & Algorithms Manual

Comprehensive Reference Guide

Computer Science Engineering

March 2026

Contents

1 Asymptotic Analysis (Big O) 3
1.1 Complexity Cheat Sheet . . . 3

2 1. Arrays (Python Lists) 4

3 2. Searching Algorithms 4
3.1 Binary Search — $O(\log n)$ . . . 4

4 3. Sorting Algorithms 5
4.1 Merge Sort — $O(n \log n)$ . . . 5
4.2 Quick Sort — $O(n \log n)$ avg, $O(n^2)$ worst . . . 6

5 4. Linked Lists 7
5.1 Singly Linked List . . . 7
5.2 Doubly Linked List — All Operations . . . 7

6 5. Stacks (LIFO) 9

7 6. Queues (FIFO) 10

8 7. Deques (Double-Ended Queue) 10

9 8. Heaps (Min-Heap) 11

10 9. Hash Tables 12

11 10. Binary Search Trees (BST) 13
11.1 Traversals and Notations . . . 14

12 11. Graphs 14
12.1 Representations . . . 14
12.1.1 Adjacency Matrix . . . 14
12.1.2 Adjacency List . . . 14
12.2 Dijkstra's Shortest Path — $O(E \log V)$ . . . 15
12.3 Minimum Spanning Tree — Kruskal's Algorithm . . . 17

13 12. Dynamic Programming **17**
13.1 Fibonacci Sequence 18
13.2 Longest Common Subsequence (LCS) 18

1 Asymptotic Analysis (Big O)

Understanding the growth rate of algorithms is fundamental to software engineering.

- **Big O (O):** Upper bound of time/space complexity.
- **Big Theta (Θ):** Tight bound.
- **Linear Search:** $O(n)$ time complexity.
- **Binary Search:** $O(\log n)$; requires a sorted array.

1.1 Complexity Cheat Sheet

Structure / Algorithm	**Access**	**Search**	**Insert**	**Delete**
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Singly Linked List	$O(n)$	$O(n)$	$O(1)$	$O(n)$
Doubly Linked List	$O(n)$	$O(n)$	$O(1)$	$O(1)^*$
Stack / Queue	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Deque	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Hash Table (avg)	—	$O(1)$	$O(1)$	$O(1)$
BST (balanced)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Heap	$O(1)^{**}$	$O(n)$	$O(\log n)$	$O(\log n)$

Algorithm	**Best**	**Average**	**Worst**
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$
Dijkstra's (heap)	$O(E \log V)$	$O(E \log V)$	$O(E \log V)$

*with direct node reference **root only

2 1. Arrays (Python Lists)

Arrays are contiguous memory blocks. Python lists are dynamic arrays that resize automatically.

```python
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target: return i
    return -1

def bubble_sort(arr): # O(n^2)
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped: break
    return arr
```

Listing 1: Array Searching and Sorting

3 2. Searching Algorithms

3.1 Binary Search — $O(\log n)$

Binary search only works on a **sorted** array. It repeatedly halves the search space by comparing the target to the middle element.

```python
# --- Iterative ---
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid            # Found     return index
        elif arr[mid] < target:
            low = mid + 1         # Search right half
        else:
            high = mid - 1        # Search left half
    return -1                     # Not found

# --- Recursive ---
def binary_search_rec(arr, target, low, high):
    if low > high:
        return -1
    mid = (low + high) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search_rec(arr, target, mid + 1, high)
    else:
        return binary_search_rec(arr, target, low, mid - 1)

```

```python
# Example
arr = [2, 5, 8, 12, 16, 23, 38, 45]
print(binary_search(arr, 23))    # Output: 5
```

Listing 2: Binary Search (Iterative & Recursive)

How it works step-by-step (searching for 23 in the array above):

1. `low=0, high=7` ⇒ `mid=3`, `arr[3]=12 < 23` ⇒ search right
2. `low=4, high=7` ⇒ `mid=5`, `arr[5]=23 == 23` ⇒ **Found!**

4 3. Sorting Algorithms

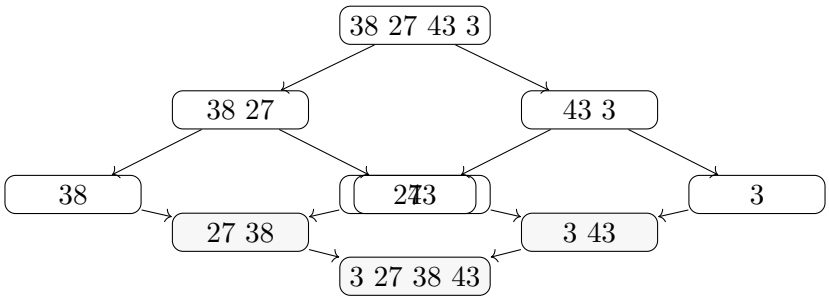
4.1 Merge Sort — $O(n \log n)$

Merge Sort uses **Divide and Conquer**. It splits the array in half recursively until single elements remain, then **merges** them back in sorted order. Space complexity: $O(n)$.

```python
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    # --- Divide ---
    mid = len(arr) // 2
    left  = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    # --- Conquer (Merge) ---
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    # Compare elements from both halves
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    # Append remaining elements
    result.extend(left[i:])
    result.extend(right[j:])
    return result

# Example
arr = [38, 27, 43, 3, 9, 82, 10]
print(merge_sort(arr))  # [3, 9, 10, 27, 38, 43, 82]
```

Listing 3: Merge Sort

Trace for `[38, 27, 43, 3]`:



4.2 Quick Sort — $O(n \log n)$ avg, $O(n^2)$ worst

Quick Sort picks a **pivot**, partitions elements smaller than pivot to the left and larger to the right, then recursively sorts each partition. Space: $O(\log n)$ (in-place).

```python
def quick_sort(arr, low, high):
    if low < high:
        pivot_idx = partition(arr, low, high)
        quick_sort(arr, low, pivot_idx - 1)   # Sort left
        quick_sort(arr, pivot_idx + 1, high)  # Sort right

def partition(arr, low, high):
    pivot = arr[high]           # Last element as pivot
    i = low - 1                # Pointer for smaller elements
    for j in range(low, high):
        if arr[j] <= pivot:   # Current element <= pivot?
            i += 1
            arr[i], arr[j] = arr[j], arr[i]  # Swap
    # Place pivot in correct position
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

# Example
arr = [10, 7, 8, 9, 1, 5]
quick_sort(arr, 0, len(arr) - 1)
print(arr)  # [1, 5, 7, 8, 9, 10]
```

Listing 4: Quick Sort

Partition trace for `[10, 7, 8, 9, 1, 5]`, pivot = `5`:

Step	Array state	Action
Start	`[10, 7, 8, 9, 1, 5]`	pivot=5, i=-1
j=4	`[1, 7, 8, 9, 10, 5]`	arr[4]=1 $\leq$ 5, swap with i=0
End	`[1, 5, 8, 9, 10, 7]`	place pivot at i+1=1

When to use which sort?

- **Merge Sort:** Prefer when stability matters or working with linked lists.
- **Quick Sort:** Prefer for in-place, cache-friendly sorting on arrays.
- **Worst case for Quick Sort:** Already sorted arrays with last-element pivot.

5 4. Linked Lists

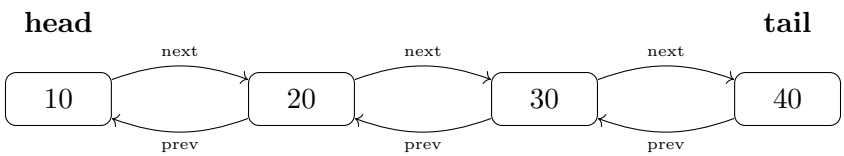
5.1 Singly Linked List

```python
class Node:
    def __init__(self, data=None):
        self.data, self.next = data, None

class LinkedList:
    def __init__(self):
        self.head = self.tail = None

    def append(self, data): # O(1) with tail
        new_node = Node(data)
        if not self.head:
            self.head = self.tail = new_node
            return
        self.tail.next = new_node
        self.tail = new_node

    def delete(self, target): # O(n)
        if self.head is None: return
        if self.head.data == target:
            self.head = self.head.next
            if self.head is None: self.tail = None
            return
        curr = self.head
        while curr.next:
            if curr.next.data == target:
                if curr.next == self.tail:
                    self.tail = curr
                curr.next = curr.next.next
                return
            curr = curr.next
```

Listing 5: Singly Linked List (linkedlist.py)

5.2 Doubly Linked List — All Operations

In a doubly linked list, every node holds a pointer to **both** the next and previous nodes, enabling $O(1)$ deletion when a node reference is available.




```python
class DNode:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None
```

```python

class DoublyLinkedList:
    def __init__(self):
        self.head = self.tail = None

    # --- ADD (append to end) --- O(1)
    def add(self, data):
        node = DNode(data)
        if not self.head:
            self.head = self.tail = node
        else:
            node.prev = self.tail
            self.tail.next = node
            self.tail = node

    # --- ADD AT FRONT --- O(1)
    def add_front(self, data):
        node = DNode(data)
        if not self.head:
            self.head = self.tail = node
        else:
            node.next = self.head
            self.head.prev = node
            self.head = node

    # --- DELETE by value --- O(n)
    def delete(self, target):
        curr = self.head
        while curr:
            if curr.data == target:
                if curr.prev:               # Not head
                    curr.prev.next = curr.next
                else:                       # Is head
                    self.head = curr.next
                if curr.next:               # Not tail
                    curr.next.prev = curr.prev
                else:                       # Is tail
                    self.tail = curr.prev
                return True                 # Deleted
            curr = curr.next
        return False                        # Not found

    # --- UPDATE first occurrence --- O(n)
    def update(self, old_val, new_val):
        curr = self.head
        while curr:
            if curr.data == old_val:
                curr.data = new_val
                return True
            curr = curr.next
        return False

    # --- SEARCH --- O(n)
```

```python
    def search(self, target):
        curr = self.head
        index = 0
        while curr:
            if curr.data == target:
                return index              # Return position
            curr = curr.next
            index += 1
        return -1                         # Not found

    # --- TRAVERSE forward --- O(n)
    def traverse_forward(self):
        result = []
        curr = self.head
        while curr:
            result.append(curr.data)
            curr = curr.next
        return result                     # e.g. [10, 20, 30]

    # --- TRAVERSE backward --- O(n)
    def traverse_backward(self):
        result = []
        curr = self.tail
        while curr:
            result.append(curr.data)
            curr = curr.prev
        return result                     # e.g. [30, 20, 10]

# --- Usage ---
dll = DoublyLinkedList()
dll.add(10); dll.add(20); dll.add(30)
print(dll.traverse_forward())     # [10, 20, 30]
print(dll.search(20))             # 1
dll.update(20, 99)
print(dll.traverse_forward())     # [10, 99, 30]
dll.delete(99)
print(dll.traverse_forward())     # [10, 30]
print(dll.traverse_backward())    # [30, 10]
```

Listing 6: Doubly Linked List — Add / Delete / Update / Search / Traverse

6 5. Stacks (LIFO)

```python
class Stack:
    def __init__(self):
        self.top = None
        self._size = 0

    def push(self, value): # O(1)
        node = Node(value)
        node.next = self.top
        self.top = node
```

```python
        self._size += 1

    def pop(self): # O(1)
        if self.is_empty(): raise IndexError("empty stack")
        data = self.top.data
        self.top = self.top.next
        self._size -= 1
        return data

    def peek(self):
        if self.is_empty(): raise IndexError("empty stack")
        return self.top.data

    def is_empty(self): return self.top is None
    def size(self): return self._size
```

Listing 7: Stack via Linked List

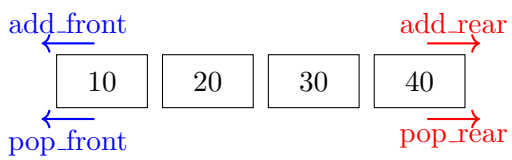
7 6. Queues (FIFO)

```python
class Queue:
    def __init__(self):
        self.front = self.rear = None

    def enqueue(self, toAdd): # O(1)
        node = Node(toAdd)
        if self.front is None:
            self.front = self.rear = node
        else:
            self.rear.next = node
            self.rear = node

    def dequeue(self): # O(1)
        if self.is_empty(): raise IndexError("empty queue")
        data = self.front.data
        self.front = self.front.next
        if self.front is None: self.rear = None
        return data

    def is_empty(self): return self.front is None
```

Listing 8: Queue via Linked List

8 7. Deques (Double-Ended Queue)

A **Deque** (pronounced "deck") allows insertion and deletion at **both** the front and rear in $O(1)$. It generalises both Stacks and Queues.



```python
class Deque:
    def __init__(self):
        self.dll = DoublyLinkedList()   # Reuse from Section 4

    # --- Add ---
    def add_front(self, data): self.dll.add_front(data)  # O(1)
    def add_rear(self, data):  self.dll.add(data)        # O(1)

    # --- Remove ---
    def pop_front(self): # O(1)
        if self.is_empty(): raise IndexError("empty deque")
        data = self.dll.head.data
        self.dll.delete(data)
        return data

    def pop_rear(self): # O(1)
        if self.is_empty(): raise IndexError("empty deque")
        data = self.dll.tail.data
        self.dll.delete(data)
        return data

    # --- Peek ---
    def peek_front(self): return self.dll.head.data if self.dll.head else None
    def peek_rear(self):  return self.dll.tail.data if self.dll.tail else None

    def is_empty(self): return self.dll.head is None

# --- Usage ---
dq = Deque()
dq.add_rear(10); dq.add_rear(20); dq.add_rear(30)
dq.add_front(5)
# Deque: 5 <-> 10 <-> 20 <-> 30
print(dq.pop_front())   # 5
print(dq.pop_rear())    # 30
# Deque: 10 <-> 20
```

Listing 9: Deque using Doubly Linked List

Use Cases:

- Sliding window problems (monotonic deque)
- Undo/Redo functionality
- Palindrome checking
- Browser history (back/forward)

9 8. Heaps (Min-Heap)

```python
class Heap:
    def __init__(self):
```

```
        self.heap = []

    def insert(self, m): # O(log n)
        self.heap.append(m)
        self._heapify_up(len(self.heap) - 1)

    def _heapify_up(self, i):
        while i > 0:
            par = (i - 1) // 2
            if self.heap[i] < self.heap[par]:
                self.heap[i], self.heap[par] = self.heap[par], self.heap[i]
                i = par
            else: break

    def delete(self): # O(log n)     removes root (minimum)
        if not self.heap: raise IndexError("empty heap")
        self.heap[0], self.heap[-1] = self.heap[-1], self.heap[0]
        val = self.heap.pop()
        self._heapify_down(0)
        return val

    def _heapify_down(self, i):
        n = len(self.heap)
        while True:
            smallest, left, right = i, 2*i+1, 2*i+2
            if left < n and self.heap[left] < self.heap[smallest]:
                smallest = left
            if right < n and self.heap[right] < self.heap[smallest]:
                smallest = right
            if smallest == i: break
            self.heap[i], self.heap[smallest] = self.heap[smallest], self.heap[i]
            i = smallest
```

Listing 10: Heap Implementation

10 9. Hash Tables

A hash table maps **keys** to **indices** via a hash function: index $= h(\text{key})\%\text{capacity}$. **Collision resolution:** *Chaining* stores multiple entries in a list at each bucket; *Open Addressing* probes for the next empty slot. **Rehashing** triggers when load factor $= n/\text{capacity} > 0.7$.

```
class HashTable:
    def __init__(self, capacity=8):
        self.capacity = capacity
        self.size = 0
        self.buckets = [[] for _ in range(self.capacity)]

    # --- Hash Function ---
    def _hash(self, key):
        return hash(key) % self.capacity

```

```python
    # --- INSERT / UPDATE --- O(1) avg
    def insert(self, key, value):
        idx = self._hash(key)
        for pair in self.buckets[idx]:
            if pair[0] == key:          # Key exists -> update
                pair[1] = value
                return
        self.buckets[idx].append([key, value])
        self.size += 1
        if self.size / self.capacity > 0.7:  # Load factor check
            self._rehash()

    # --- SEARCH --- O(1) avg
    def get(self, key):
        idx = self._hash(key)
        for pair in self.buckets[idx]:
            if pair[0] == key:
                return pair[1]
        return None                      # Key not found

    # --- DELETE --- O(1) avg
    def delete(self, key):
        idx = self._hash(key)
        for i, pair in enumerate(self.buckets[idx]):
            if pair[0] == key:
                self.buckets[idx].pop(i)
                self.size -= 1
                return True
        return False

    # --- REHASH --- O(n)
    def _rehash(self):
        old_buckets = self.buckets
        self.capacity *= 2
        self.buckets = [[] for _ in range(self.capacity)]
        self.size = 0
        for bucket in old_buckets:
            for key, value in bucket:
                self.insert(key, value)

# --- Usage ---
ht = HashTable()
ht.insert("name", "Alice")
ht.insert("age", 22)
print(ht.get("name"))    # Alice
ht.delete("age")
print(ht.get("age"))     # None
```

Listing 11: Hash Table with Chaining & Rehashing

11 10. Binary Search Trees (BST)

In a BST: left child < root < right child.

11.1 Traversals and Notations

- **In-order (L-Root-R):** Produces sorted order; **Infix** notation. Display node 2nd time seen.
- **Pre-order (Root-L-R): Prefix** notation. Display node 1st time seen.
- **Post-order (L-R-Root): Postfix** notation. Display node 3rd time seen.

```python
def _inorder(self, node, traversal):
    if node:
        self._inorder(node.left, traversal)
        traversal.append(node.data)
        self._inorder(node.right, traversal)

def level_order(self): # BFS - O(n)
    if not self.root: return []
    result, queue = [], [self.root]
    while queue:
        node = queue.pop(0)
        result.append(node.data)
        if node.left: queue.append(node.left)
        if node.right: queue.append(node.right)
    return result
```

Listing 12: BST Traversals

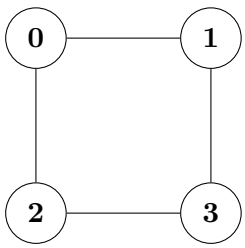
12 11. Graphs

12.1 Representations

12.1.1 Adjacency Matrix

A 2D array where `matrix[i][j] = 1` means there is an edge from vertex i to vertex j. Space: $O(V^2)$. Fast edge lookup $O(1)$, but wastes space for sparse graphs.

Graph



	0	**1**	**2**	**3**
0	0	1	1	0
1	1	0	0	1
2	1	0	0	1
3	0	1	1	0

12.1.2 Adjacency List

Each vertex stores a list of its neighbours. Space: $O(V + E)$. Efficient for sparse graphs. Most practical graph algorithms use this.

$$\begin{aligned} 0: &\quad [1, 2] \\ 1: &\quad [0, 3] \\ 2: &\quad [0, 3] \\ 3: &\quad [1, 2] \end{aligned}$$

```python
# --- Adjacency List ---
graph_list = {
    0: [1, 2],
    1: [0, 3],
    2: [0, 3],
    3: [1, 2]
}

# --- Adjacency Matrix ---
V = 4
graph_matrix = [[0]*V for _ in range(V)]
edges = [(0,1),(0,2),(1,3),(2,3)]
for u, v in edges:
    graph_matrix[u][v] = 1
    graph_matrix[v][u] = 1   # Undirected

# --- BFS (uses Queue) --- O(V + E)
from collections import deque
def bfs(graph, start):
    visited, queue = set([start]), deque([start])
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return order

# --- DFS (uses Stack / recursion) --- O(V + E)
def dfs(graph, node, visited=None):
    if visited is None: visited = set()
    visited.add(node)
    result = [node]
    for neighbor in graph[node]:
        if neighbor not in visited:
            result += dfs(graph, neighbor, visited)
    return result

print(bfs(graph_list, 0))   # [0, 1, 2, 3]
print(dfs(graph_list, 0))   # [0, 1, 3, 2]
```

Listing 13: Graph Representation in Python

12.2 Dijkstra's Shortest Path — $O(E \log V)$

Finds the shortest path from a **source** vertex to all others in a weighted graph with **non-negative** edge weights. Uses a **min-heap** (priority queue).

Algorithm:

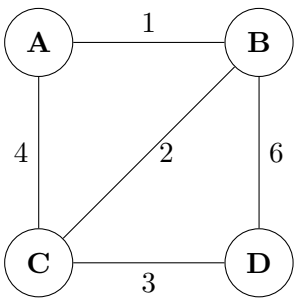
1. Initialise `dist[source] = 0`, all others $= \infty$.
2. Push `(0, source)` into min-heap.

3. Pop the vertex with smallest distance. For each neighbour, if `dist[u] + w < dist[v]`, update and push.

4. Repeat until heap is empty.

```python
import heapq

def dijkstra(graph, source):
    """
    graph: dict of {node: [(neighbor, weight), ...]}
    Returns: dist dict with shortest distance from source
    """
    dist = {node: float('inf') for node in graph}
    dist[source] = 0
    min_heap = [(0, source)]   # (distance, node)

    while min_heap:
        curr_dist, u = heapq.heappop(min_heap)

        # Skip if we already found a shorter path
        if curr_dist > dist[u]:
            continue

        for v, weight in graph[u]:
            new_dist = dist[u] + weight
            if new_dist < dist[v]:        # Relaxation
                dist[v] = new_dist
                heapq.heappush(min_heap, (new_dist, v))

    return dist

# --- Example ---
graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('A', 1), ('C', 2), ('D', 6)],
    'C': [('A', 4), ('B', 2), ('D', 3)],
    'D': [('B', 6), ('C', 3)]
}
print(dijkstra(graph, 'A'))
# {'A': 0, 'B': 1, 'C': 3, 'D': 6}
```

Listing 14: Dijkstra's Algorithm



Vertex	Shortest dist from A
A	0
B	1
C	3 (via B)
D	6 (via B→C)

12.3 Minimum Spanning Tree — Kruskal's Algorithm

An **MST** connects all vertices with the minimum total edge weight and no cycles. Kruskal's greedily picks the cheapest edge that doesn't form a cycle, using a **Union-Find** structure.

```python
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank   = [0] * n

    def find(self, x):                      # Path compression
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):                  # Union by rank
        px, py = self.find(x), self.find(y)
        if px == py: return False           # Already connected (cycle!)
        if self.rank[px] < self.rank[py]: px, py = py, px
        self.parent[py] = px
        if self.rank[px] == self.rank[py]: self.rank[px] += 1
        return True

def kruskal(n, edges):
    """
    n: number of vertices
    edges: list of (weight, u, v)
    Returns: list of MST edges and total weight
    """
    edges.sort()                            # Sort by weight
    uf = UnionFind(n)
    mst, total = [], 0
    for weight, u, v in edges:
        if uf.union(u, v):                  # No cycle -> add edge
            mst.append((u, v, weight))
            total += weight
    return mst, total

# --- Example (same graph as Dijkstra above: A=0, B=1, C=2, D=3) ---
edges = [(1,0,1),(4,0,2),(2,1,2),(6,1,3),(3,2,3)]  # (weight, u, v)
mst, total = kruskal(4, edges)
print(mst)    # [(0,1,1), (1,2,2), (2,3,3)]  -> edges A-B, B-C, C-D
print(total)  # 6
```

Listing 15: Kruskal's MST with Union-Find

13 12. Dynamic Programming

Dynamic Programming (DP) solves problems by breaking them into **overlapping subproblems** and storing results to avoid recomputation.

- **Memoisation (Top-Down):** Recursive + cache.
- **Tabulation (Bottom-Up):** Iterative table fill.

13.1 Fibonacci Sequence

$F(n) = F(n-1) + F(n-2)$, $F(0) = 0$, $F(1) = 1$.

```python
# --- Naive Recursion: O(2^n) --- SLOW
def fib_naive(n):
    if n <= 1: return n
    return fib_naive(n-1) + fib_naive(n-2)

# --- Memoisation (Top-Down): O(n) ---
def fib_memo(n, memo={}):
    if n in memo: return memo[n]
    if n <= 1: return n
    memo[n] = fib_memo(n-1, memo) + fib_memo(n-2, memo)
    return memo[n]

# --- Tabulation (Bottom-Up): O(n) time, O(n) space ---
def fib_tab(n):
    if n <= 1: return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]

# --- Space-Optimised: O(n) time, O(1) space ---
def fib_opt(n):
    if n <= 1: return n
    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b

print(fib_tab(10))   # 55
print(fib_opt(10))   # 55
```

Listing 16: Fibonacci — Naive vs Memoisation vs Tabulation

Subproblem table for $n = 6$:

i	0	1	2	3	4	5	6
$F(i)$	0	1	1	2	3	5	8

13.2 Longest Common Subsequence (LCS)

Given two strings X and Y, find the length of their longest common subsequence (not necessarily contiguous).

Recurrence:

$$\text{dp}[i][j] = \begin{cases} \text{dp}[i-1][j-1] + 1 & \text{if } X[i-1] = Y[j-1] \\ \max(\text{dp}[i-1][j],\ \text{dp}[i][j-1]) & \text{otherwise} \end{cases}$$

```python
def lcs(X, Y):
    m, n = len(X), len(Y)
    # Build (m+1) x (n+1) DP table
```

```python
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if X[i-1] == Y[j-1]:            # Characters match
                dp[i][j] = dp[i-1][j-1] + 1
            else:                            # Take the best of skip
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    # --- Backtrack to find the actual subsequence ---
    lcs_str, i, j = "", m, n
    while i > 0 and j > 0:
        if X[i-1] == Y[j-1]:
            lcs_str = X[i-1] + lcs_str
            i -= 1; j -= 1
        elif dp[i-1][j] > dp[i][j-1]:
            i -= 1
        else:
            j -= 1

    return dp[m][n], lcs_str

# --- Example ---
X, Y = "ABCBDAB", "BDCABA"
length, subseq = lcs(X, Y)
print(f"Length: {length}")      # 4
print(f"LCS: {subseq}")         # BCBA (or BDAB     multiple valid
    answers)
```

Listing 17: LCS — Tabulation

DP Table for $X =$ ABCB, $Y =$ BCB:

		""	**B**	**C**	**B**
	""	0	0	0	0
A		0	0	0	0
B		0	1	1	1
C		0	1	2	2
B		0	1	2	**3**

LCS length = 3 (BCB).